

Project 7 实验报告

陳日康 22336049

一、程序功能简要说明

这是一个简单的人名哈希表的实现，其中包括了初始化姓名表、创建哈希表、显示姓名表、显示哈希表、查找人名以及交互界面等功能。下面是程序的功能简明说明：

1. 初始化姓名表 (InitName):

预先定义了30个姓名，每个姓名包括拼音和对应的ASCII总和。

2. 创建哈希表 (CreateHash):

使用除留余数法和线性探测法解决哈希冲突，将姓名表中的姓名插入哈希表。当哈希表发生冲突时，通过线性探测法寻找下一个可用位置。如果哈希表已满，则输出提示信息。

3. 显示姓名表 (DisplayName):

打印姓名表的地址、姓名、关键词（ASCII总和）。

4. 显示哈希表 (DisplayHash):

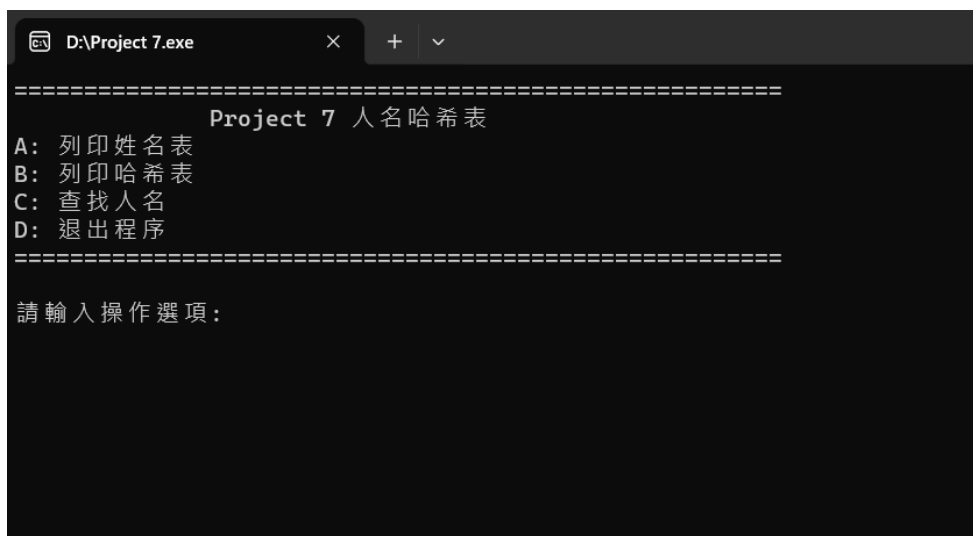
打印哈希表的地址、姓名、关键词、搜索长度和计算平均查找长度（ASL）并输出。

5. 查找人名 (FindName):

用户输入要查找的名字。通过哈希表查找名字对应的地址，并输出姓名、关键词、地址以及查找长度。如果不存在，则输出相应提示。

程序通过哈希表实现了对人名的快速查找，并提供了简单的交互界面。用户可以选择打印姓名表、打印哈希表、查找人名或退出程序。

二、程序运行截图，包括功能演示、部分实际运行结果演示、命令行或交互式界面结果等



```
D:\Project 7.exe
=====
Project 7 人名哈希表
A: 列印姓名表
B: 列印哈希表
C: 查找人名
D: 退出程序
=====
請輸入操作選項:
```

在程序开始界面，程序会让用户输入所需的操作，选项A是列印姓名表，选项B是列印哈希表，选项C是查找人名功能，而选项D是退出程序。

請輸入操作選項：A

姓名表如下：

地址：0	姓名：Cheniathong	關鍵字：1128
地址：1	姓名：Chenchuan	關鍵字：909
地址：2	姓名：Hangbohao	關鍵字：903
地址：3	姓名：Chenyonghui	關鍵字：1153
地址：4	姓名：Chengang	關鍵字：795
地址：5	姓名：Zhoujielun	關鍵字：1069
地址：6	姓名：Ngcheokian	關鍵字：1015
地址：7	姓名：Huangjianqiao	關鍵字：1343
地址：8	姓名：Guizixuan	關鍵字：964
地址：9	姓名：Chenrikang	關鍵字：1018
地址：10	姓名：Huangyushuo	關鍵字：1184
地址：11	姓名：Wangshaoquan	關鍵字：1261
地址：12	姓名：Wuyangtianlang	關鍵字：1481
地址：13	姓名：Wuzhuoxin	關鍵字：993
地址：14	姓名：Liuxionghao	關鍵字：1159
地址：15	姓名：Zhuxun	關鍵字：658
地址：16	姓名：Luyao	關鍵字：522
地址：17	姓名：Eriktenhag	關鍵字：1026
地址：18	姓名：Maddison	關鍵字：815
地址：19	姓名：Huangyongming	關鍵字：1371
地址：20	姓名：Sonheungmin	關鍵字：1163
地址：21	姓名：Fandewen	關鍵字：808
地址：22	姓名：Yangyue	關鍵字：738
地址：23	姓名：Zhanghongkai	關鍵字：1241
地址：24	姓名：Romero	關鍵字：628
地址：25	姓名：Richa	關鍵字：487
地址：26	姓名：Liangjingqian	關鍵字：1340
地址：27	姓名：Wulei	關鍵字：518
地址：28	姓名：Lihaorong	關鍵字：931
地址：29	姓名：Bentancur	關鍵字：930

=====

請輸入操作選項：|

若用户选择选项 A，则程序会打印出姓名表，当中包括地址、在程序预先定义了30个的姓名还有关键字（ASCII总和）。

請輸入操作選項：B

地址	姓名	關鍵字	搜索長度
0	Cheniathong	1128	1
1	Zhuxun	658	2
2	Wulei	518	2
3		0	0
4		0	0
5	Luyao	522	1
6	Wuzhuoxin	993	1
7		0	0
8	Huangyongming	1371	1
9	Huangyushuo	1184	1
10	Hangbohao	903	1
11	Fandewen	808	3
12		0	0
13		0	0
14		0	0
15		0	0
16	Chenchuan	909	1
17	Maddison	815	2
18	Romero	628	2
19	Zhanghongkai	1241	1
20	Richa	487	4
21		0	0
22		0	0
23		0	0
24	Guizixuan	964	1
25	Chenyonghui	1153	1
26	Wuyangtianlang	1481	3
27	Huangjianqiao	1343	1
28	Ngcheokian	1015	1
29	Liangjingqian	1340	6
30		0	0
31	Chenrikang	1018	1
32	Liuxionghao	1159	2
33	Yangyue	738	1
34		0	0
35	Zhoujielun	1069	1
36	Sonheungmin	1163	2
37	Bentancur	930	1
38	Lihaorong	931	1
39	Wangshaoquan	1261	1
40	Eriktenhag	1026	2
41		0	0
42		0	0
43	Chengang	795	1
44		0	0
45		0	0
46		0	0
47		0	0
48		0	0
49		0	0

平均查找長度：ASL(30) = 1.63333

=====

若用户选择操作 **B**，程序会打印出哈希表的地址、姓名、关键词，还会计算出搜索长度 **ASL**。

```
請輸入操作選項：C  
請輸入想要查找的名字：|
```

```
請輸入操作選項：C  
請輸入想要查找的名字：Chenrikang  
姓名：Chenrikang      關鍵字：1018      地址：31
```

若用户选择操作 **C**，程序会先要求用户输入所需要查找的名字。如果用户输入的名字是在姓名表里，则程序会通过哈希表查找名字对应的地址，并输出姓名、关键词、地址。

```
請輸入操作選項：C  
請輸入想要查找的名字：Chenrihang  
你輸入的名字不在姓名表中！
```

如果用户选择操作 **C** 且如果输入的名字不在姓名表里，则程序会输出“你输入的姓名不在姓名表中！”。

```
請輸入操作選項：D  
  
-----  
Process exited after 2.096 seconds with return value 0  
請按任意鍵繼續 . . . |
```

如果用户选择操作 **D**，则程序结束运行。

三、部分关键代码以及说明

1. 姓名表和哈希表结构体

```
typedef struct {  
    char *name;           //姓名表  
    int hashCode;         //名字的拼音  
} NAME;  
//拼音所对应的ASCII和
```

NAME 结构体用于表示姓名表中的一项，包括名字的拼音和对应的 ASCII 和。**char *name** 存储名字的拼音，这里直接使用字符串指针。**int hashCode** 存储拼音对应的 ASCII 和。

2. 哈希表结构体

```
typedef struct {           //哈希表
    char *name;           //名字的拼音
    int key;               //拼音所对应的ASCII总和，即关键词
    int si;                //查找长度
} HASH;
```

HASH 结构体用于表示哈希表中的一项，包括名字的拼音、关键词和查找长度。`char *name` 存储名字的拼音。`int key` 存储拼音对应的 ASCII 总和，即关键词。`int si` 存储查找长度，表示在查找该项时需要经过的哈希表位置个数。

3. 初始化姓名表

```
void InitName() { //姓名表的初始化
    Name[0].name = "Cheniathong";
    Name[1].name = "Chenchuan";
    Name[2].name = "Hangbohao";
    Name[3].name = "Chenyonghui";
    Name[4].name = "Chengang";
    Name[5].name = "Zhoujielun";
    Name[6].name = "Ngcheokian";
    Name[7].name = "Huangjianqiao";
    Name[8].name = "Guizixuan";
    Name[9].name = "Chenrikang";
    Name[10].name = "Huangyushuo";
    Name[11].name = "Wangshaoquan";
    Name[12].name = "Wuyangtianlang";
    Name[13].name = "Wuzhuoxin";
    Name[14].name = "Liuxionghao";
    Name[15].name = "Zhuxun";
    Name[16].name = "Luyao";
    Name[17].name = "Eriktenhag";
    Name[18].name = "Maddison";
    Name[19].name = "Huangyongming";
    Name[20].name = "Sonheungmin";
    Name[21].name = "Fandewen";
    Name[22].name = "Yangyue";
    Name[23].name = "Zhanghongkai";
    Name[24].name = "Romero";
    Name[25].name = "Richa";
    Name[26].name = "Liangjingqian";
    Name[27].name = "Wulei";
    Name[28].name = "Lihaorong";
    Name[29].name = "Bentancur";

    for (i=0; i<NAME_LEN; i++) { //将字符串的各个字符所对应的ASCII码相加，所得的整数做
        //为哈希表的关键词
    }
}
```

```

        Name[i].hashCode = getHashCode(Name[i].name);
    }
}

```

`InitName` 函数用于初始化姓名表中的数据和将字符串的各个字符所对应的ASCII码相加，所得的整数做为哈希表的关键词。这里预先定义了需填入哈希表的30个名字。

4. 创建哈希表

```

void CreateHash() {                                //建立哈希表
    for (i=0; i<HASH_LEN; i++) {                  //清空哈希表，未经此操作将储存空数
据
        Hash[i].name = "\0";
        Hash[i].key = 0;
        Hash[i].si = 0;
    }
    for (i=0; i<NAME_LEN; i++) {
        int si = 1;                                //查找长度默认为1
        int adr = (Name[i].hashCode) % P;          //除留余数法H(name)=name%P，除数
为P=47

        if (Hash[adr].si != 0) {                  //如果冲突，使用线性探测法处理冲突
            int currAddr = adr;
            //从冲突下一个位置开始探测，到达最后一个再从第一个开始
            do {
                si++;                                // 查找长度+1
                adr = (adr + 1) % HASH_LEN;
            } while (Hash[adr].si != 0 && adr != currAddr);

            // 直到回到当前位置时还没找到，说明哈希表已经满了
            if (adr == currAddr) {
                cout << "哈希表已满" << endl;
                return;
            }
        }
        Hash[adr].key = Name[i].hashCode;
        Hash[adr].name = Name[i].name;
        Hash[adr].si = si;
    }
}

```

函数 `CreateHash` 初始化哈希表 (`Hash`) 并将姓名表 (`Name`) 中的姓名插入其中。使用姓名的 ASCII 之和作为哈希表中的初始地址。在发生冲突时，使用线性探测法找到下一个可用的槽位。函数计算每次插入的搜索长度 (`si`) 并将其存储在哈希表中。

5. 显示姓名表

```

void DisplayName() { //显示姓名表
    cout << endl;
    cout << "姓名表如下：" << endl;
    for (i = 0; i < NAME_LEN; i++) {
        cout << "地址：" << left << setw(10) << i << "姓名：" << left << setw(20)
        << Name[i].name << "关键词：" << left << setw(10) << Name[i].hashCode << endl;
    }
    cout << endl;
    cout << "======" << endl;
}

```

`DisplayName` 函数用于打印姓名表，显示每一项的地址、姓名和关键词。

6. 显示哈希表

```

void DisplayHash() { // 显示哈希表
    float asl = 0.0;
    cout << endl << endl << " 地址 \t\t 姓名 \t\t 关键词 \t 搜索长度" << endl;
    //显示的格式
    for (i=0; i<HASH_LEN; i++) {
        printf("%2d %18s \t\t %d \t\t %d\n", i, Hash[i].name, Hash[i].key,
        Hash[i].si);
        asl += Hash[i].si;
    }
    asl /= NAME_LEN; //求得ASL
    cout << endl << endl;
    cout << "平均查找长度：ASL(" << NAME_LEN << ") = " << asl << endl;
    cout << endl;
    cout << "======" << endl;
}

```

函数 `DisplayHash` 显示哈希表的内容，包括地址、姓名、关键词以及每个条目的搜索长度。计算并打印哈希表中所有条目的平均搜索长度 (ASL)。

7. 查找名字

```

// 查询
void FindName () {
    char name[20] = {0};
    int hashCode = 0, si = 1;
    cout << endl;
    cout << "请输入想要查找的名字：" ;
    cin >> name;
    getchar();

    hashCode = gethashCode(name); //求出姓名的拼音所对应的ASCII作为关键词
    int adr = hashCode % P; //除留余数法去地址
    int j = 0;

    // 如果hash地址为空，则直接认为不存在
    if (Hash[adr].key == 0) {

```

```

        cout << endl;
        cout << "你输入的名字不在姓名表中！" << endl;
        cout << endl;
        cout << "=====" <<
endl;
        return;
    }

    // 如果hashCode和name都相等
    if (Hash[adr].key == hashCode && 0 == strcmp(Hash[adr].name, name)) {
        cout << endl;
        cout << "姓名: " << left << setw(17) << Hash[adr].name << "关键词: " <<
left << setw(12) << hashCode << "地址: " << left << setw(15) << adr;
        cout << endl << endl;
        cout << "=====" <<
endl;
    }

    // 如果不相等，则进行线性探测搜索
    else {
        int currAddr = adr;
        //从冲突下一个位置开始探测,到达最后一个再从第一个开始
        do {
            si++; // 查找长度+1
            adr = (adr + 1) % HASH_LEN;
            // 如果找到，直接break跳出循环
            if (Hash[adr].key == hashCode && 0 == strcmp(Hash[adr].name, name)) {
                cout << endl;
                cout << "姓名: " << left << setw(17) << Hash[adr].name << "关键词: "
<< left << setw(12) << hashCode << "地址: " << left << setw(12) << adr << "查找长
度为: " << left << setw(15) << si;
                cout << endl << endl;
                cout << "====="
<< endl;
                break;
            }
        } while (adr != currAddr);

        // 直到回到当前位置时还没找到，说明不存在
        if (adr == currAddr) {
            cout << endl;
            cout << "你输入的名字不在姓名表中！" << endl;
            cout << endl;
            cout << "=====" <<
endl;
        }
    }
}

```

函数 `FindName` 会根据用户输入要搜索的姓名，计算输入姓名的哈希码，并确定在哈希表中的初始地址。如果在计算的地址找到姓名，它打印详细信息（姓名、关键词、地址）。如果未找到，则使用线性探测搜索姓名，并打印详细信息以及搜索长度。

四、程序运行方式简要说明

程序首先会初始化姓名表，姓名表中包含了30个人名以及它们对应的拼音所对应的ASCII和。接着，程序会创建一个哈希表，使用除留余数法和线性探测法来处理冲突，将姓名表中的数据插入到哈希表中。初始化和哈希表创建完成后，用户将看到一个交互界面，其中有四个选项：**A**用于打印姓名表，显示每一项的地址、姓名和关键词；**B**用于打印哈希表，显示每一项的地址、姓名、关键词和查找长度；**C**用于查找人名，用户需要输入想要查找的姓名的拼音，程序将显示查找结果，包括姓名、关键词、地址和查找长度；最后的选项**D**是退出程序。用户可以根据需要选择不同的操作，直到选择退出为止。

五、实验感想

在完成这个哈希表实验的过程中，我对哈希表的原理和实现方法有了更深刻的理解。通过实际编写代码，我学到了如何使用哈希函数将数据映射到哈希表中，并了解了解决冲突的方法，本次实验中使用的是线性探测法。这种方法通过在哈希表中顺序查找下一个可用的位置来解决冲突，从而保证数据能够被正确插入。在实验中，我还学到了如何设计合适的哈希函数，以确保数据在哈希表中均匀分布，减少冲突的发生。通过调整哈希函数的设计，我能够影响哈希表的性能，使其更加高效。

此外，通过编写交互式的用户界面，我学到了如何设计用户友好的程序，提供清晰的操作界面，使用户能够方便地使用哈希表功能。在处理用户输入、调用相应函数以及输出结果方面，我积累了一定的编程经验。通过这次实验，我不仅加深了对哈希表的理解，还提高了编程能力，学到了一些实用的算法和数据结构的应用方法。这些知识和经验将对我今后的学习和工作产生积极的影响。